CS 4501 · ALPHA DEVELOPMENT

Cleanasaurus Activities Document

MVP validation, requirement decomposition, and AI-assisted test-driven development

Team	Nate Kim, Nate Park, Wesley Chang
Repository	github.com/natekim416/cs4501_startup (Alpha merged into `main`)
Issues	29 user stories as GitHub issues #2–#30
Date	November 5, 2026

Contents

1. **Activity 1 · MVP hypothesis check and validation analysis**: 10 testers, KPI results, decision: **Pivot** to developer clutter for CS students
2. **Activity 2 · Requirement decomposition**: core requirements and LLM integration, personas, 29 user stories with acceptance criteria, release plan, GitHub issues
3. **Activity 3 · AI-assisted TDD**: tests for every issue first, red and green runs, issue-to-test map, review log of AI-generated tests
4. **Appendix**: the Alpha running on a dev-clutter demo folder

Activity 1 · MVP Hypothesis Check & Validation Analysis

Source: validation/MVP_VALIDATION.md

Team: Nate Kim, Nate Park, Wesley Chang · **Testers:** 10 (5 CS, 5 non-CS) · **Testing:** October 30 – November 2, 2026 (Test 1 sessions, then 3 days of Test 2)

Decision: Pivot. Testers trusted the app (2 of 3 trust KPIs met, and the third close), but it didn't solve a problem they cared about (0 of 2 value KPIs met). We keep the engine and aim it at CS students' rebuildable developer clutter.

---

1. Running the validation plan

Core hypothesis: Students with cluttered laptops will let an AI clean and reorganize their files, as long as every plan is checked and previewed first.

Target population: 5 CS-related friends and classmates (C1–C5), and 5 non-CS testers (N1–N5): four family members and one non-CS student.

Change from our plan: we planned 15 testers (10 CS, 5 non-CS) and ran 10 (5 CS, 5 non-CS), so the CS results rest on fewer people.

Test 1: A 20-minute think-aloud on the tester's own laptop, with three tasks: free up space, organize files, and scan for malware.

Test 2: 3 days of unsupervised use, measured with the app's opt-in usage log. 8 of 10 opted in and sent us the exported log, and we asked the other two how often they ran it.

What each tester told us

> The percentage is how full each tester's disk was.

Tester	Who	Summary
C1	CS, 94%	Approved all 46 actions in Downloads, but said their space is in the `node_modules` of about 30 projects, which the app won't touch, and they'd run it weekly if it could clean those.
C2	CS, does ML research, 88%	Approved 36 of 38 actions because the app proved the dataset copies were identical, and wants it to clear some models they haven't loaded in a month.
C3	CS, builds iOS apps, 97%	Approved 33 of 35 actions on the Desktop, but had to drag two screenshots back by hand because undo is all-or-nothing, and said their real space is Xcode data the app won't touch.

Tester	Who	Summary
C4	CS, took the security course, 71%	Ran only 14 of 20 actions after the app flagged their own unsigned app as malware, noticed the Dino Bin frees nothing on disk, and wants a local model because file names are private.
C5	CS (graduate), makes games and music, 89%	Approved 26 of 27 actions, started trusting the app when it asked about a 1 GB drum pack, and wants it to clear Unity `Library` folders that rebuild on open.
N1	Family (dad), 93%	Approved 18 of 30 actions after unchecking every photo move and most wants to ask where a downloaded form went.
N2	Family (mom, least comfortable with technology), 78%	Watched the demo on a fake folder but wouldn't let it touch their own files.
N3	Family (sibling, law student), 96%	Approved a 41-action plan without reading it, undid everything when their essay moved, and wants a one-sentence summary and help removing games they don't play.
N4	Family (sibling, less comfortable with technology), 89%	Approved 20 of 22 actions and loved the clean Desktop, but didn't know what "unsigned" meant and wants scam checks on email attachments.
N5	Friend, pre-med student, 91%	Approved all 33 actions, loved that it remembered which essay draft to keep, and wants it to move last semester's files to an external drive.

---

2. KPI data

2.1 Quantitative

Trust KPIs ask whether people will let the app act. Value KPIs ask whether it was worth it.

Group	KPI	How we measure it	Target	All (10)	CS (5)	Non-CS (5)	Rating
Trust	First-session approval	Ran at least 80% of a plan on their own files in session 1	≥60%	**7/10 = 70%**	4/5 = 80%	3/5 = 60%	🟢
Trust	Unsafe actions	Shown to a tester, and would lose data, touch a NEVER zone, delete a non-duplicate, or quarantine an unflagged file	0	**0**	0	0	🟢
Trust	Undo rate	Actions undone ÷ actions run	≤10%	**41/361 = 11%**	0/212 = 0%	41/149 = 28%	🟡
Value	Repeat use	Ran another plan within 3 days, on their own	≥40%	**3/10 = 30%**	2/5 = 40%	1/5 = 20%	🟡
Value	Space freed	Median space reported freed ÷ median space wanted back	≥50%	**0.6 ÷ 27.5 GB ≈ 2%**	1.4 ÷ 30 GB ≈ 5%	0.3 ÷ 15 GB = 2%	🔴

Scale: 🟢 met the target · 🟡 close, within 15 points · 🔴 missed by more, or any unsafe action.

Decision rules:

- **Validated:** no trust KPI is 🔴, and every value KPI is 🟢.
- **Pivot:** no trust KPI is 🔴, but a value KPI isn't 🟢. The engine works, but it's aimed at the wrong problem or people.
- **Invalidated:** a trust KPI is 🔴. People won't let the app act, or it isn't safe.

Result: trust 2 of 3 with none 🔴, value 0 of 2, so Pivot. CS testers alone met 3 of 3 and 1 of 2, which is still Pivot. Non-CS testers alone met 2 of 3 with a 🔴 undo rate and 0 of 2, which is Invalidated for that group and one reason the pivot targets CS students.

Tester	Actions kept	Ran ≥80%?	Trust (§2.2)	Reported freed	Wanted back	Rebuildable dev clutter	Test 2 plans
C1	46/46	✅	High	1.4 GB	30 GB	38 GB	0
C2	36/38	✅	High	6.6 GB	40 GB	61 GB	1
C3	33/35	✅	Medium	0.3 GB	50 GB	57 GB	0
C4	14/20	❌ 70%	Low	0.8 GB	15 GB	9 GB	0
C5	26/27	✅	High	2.1 GB	25 GB	27 GB	1
N1	18/30	❌ 60%	Low	0.1 GB	10 GB	—	0
N2	Demo only	❌	Low	—	"no idea"	—	0
N3	41/41, all undone	✅*	Low	0 GB	50 GB	—	0
N4	20/22	✅	Medium	0.6 GB	"no idea"	—	0
N5	33/33	✅	High	0.5 GB	15 GB	—	1

* N3 approved without reading. Without N3, approval is 6/9 = 67%. All 41 undone actions came from N3's single undo, so without N3 the undo rate is 0%.

Other measurements:

- **Validator:** of 37 plan requests, 36 passed within 3 tries (24 on the first), and 1 fell back to the safe part. It caught 20 errors along the way.
- **Disk:** the FREED counter reported 12.4 GB in total, but **0 bytes were actually freed on disk**, because the Dino Bin is on the same drive.
- **Speed:** a validated plan took a median of 34 seconds.

Where CS testers' space actually is:

Category	Total	Testers who had it
Xcode build data and simulators	57 GB	1
ML model caches (Hugging Face, PyTorch)	44 GB	1
JavaScript packages (`node_modules`, npm cache)	26 GB	2
Docker images and build cache	25 GB	3

Category	Total	Testers who had it
Python environments and caches	22 GB	2
Game-engine build folders (Unity `Library/`)	18 GB	1
Total rebuildable	**192 GB** (median 38 GB per tester, range 9–61 GB)	

The same 5 testers had a median of **1.4 GB** of reported savings from duplicates, so their rebuildable clutter is **about 27× larger**.

2.2 Qualitative

Trust level, from what each tester did in session 1:

Level	What we saw	Testers
High	Ran at least 80% of a plan, then asked it to do more or said they'd let it run alone	4: C1, C2, C5, N5
Medium	Trusted the plan but would want to check it every time, or it just wasn't useful to them	2: C3, N4
Low	Held back out of distrust, reversed a plan they hadn't read, or wouldn't use their own files	4: C4, N1, N2, N3

CS testers were much more trusting than non-CS testers: 4 of 5 were Medium or High, against 2 of 5.

Themes. A theme counts if at least 4 of 10 testers, or 3 of the 5 non-CS testers, raised it unprompted.

Theme	Testers	Count
The preview and plain-English reasons are why I approved	C1–C5, N4, N5	7/10
My real space problem is somewhere the app won't go	C1–C5, N1, N3	7/10 (5/5 CS)
Undo and "nothing is gone for good" are why I approved	C1, C2, C5, N4, N5	5/10
It's too technical, or there's too much to read	N2, N3, N4	3/5 non-CS

Below the bar: questions and memory were the best part (3/10), a helper to find files or spot scams (2/10), run by itself (2/10), too slow (2/10), sort by more than file names (1/10), remove things that aren't copies (1/10), and file names are private (1/10).

---

3. Decision: Pivot

3.1 What the evidence says

Trust is validated.

- 70% ran a plan in their first session, above our 60% target, and still 67% without N3's weak approval.
- No unsafe action reached anyone, even though the validator caught 20 errors.
- The undo rate is 11%, just over our 10% target, and all of it comes from N3's single undo.

- The trust features we designed were the top reasons testers gave for approving: previews with reasons (7/10) and undo (5/10).

Value is not validated.

- Testers wanted a median of 27.5 GB back. The app reported 0.6 GB, and the real number was 0, because the Dino Bin sits on the same disk (executor.js:8).
- Only 30% used it again in Test 2.
- 7 of 10 said, unprompted, that their real space problem was somewhere the app won't go. That's by design today: git repos are NEVER zones (profile.js:49), `~/Library` is too (profile.js:11), and the scan skips `node_modules` (scanner.js:6).

Why not "Validated"? The hypothesis only asked whether people would *let* the app act. They did, but with someone sitting next to them and on low-stakes folders. That shows permission, not demand. An app that frees about 2% of the space people want doesn't have a business, however much they trust it.

Why not just "Invalidated"? Nothing about trust failed. The preview, reasons, validator, undo, and questions are the parts that work, and they carry over. What failed is the problem we aimed them at, so we change the target and keep the engine.

Limits of this evidence. Testers are friends and family, which inflates approval and trust, though value still scored low despite that bias. With n = 10, one tester moves a percentage by 10 points, and by 20 points among the 5 CS testers the pivot leans on, so treat the numbers as directional. Three days is also a short window for a problem that builds up over weeks.

3.2 What we're pivoting to

From: a general file cleaner for "students with cluttered laptops" that sorts Downloads and trashes exact duplicates.

To: Cleanasaurus for developers' laptops, starting with CS students. It reclaims space from developer clutter that can be rebuilt, and it only deletes what the validator can prove you can get back.

This is a **customer-segment pivot** (from everyone with a Mac to CS students) plus a **zoom-in pivot** (one capability, "free up space," becomes the product). The vision stays the same: tell your computer to clean itself, and trust it to do it safely.

New hypothesis:

> CS students whose laptops are over 80% full will, without us in the room, approve a validated plan that actually frees at least 10 GB of rebuildable developer clutter in their first session (≥60% of testers), and at least 40% will run it again within 30 days.

What it looks like:

> **Student:** I'm out of space. I'm done with CS 3140 and the fall hackathon, but don't touch my capstone or anything on my resume.
>
> **Dino:** I found 38 GB I can prove you can rebuild:
>
> - `node_modules` in 6 projects you haven't opened since spring: 4.1 GB. `npm ci` rebuilds each in about a minute.
> - 3 Python environments from CS 3140: 2.2 GB. Rebuilt from each `requirements.txt`.
> - Xcode build data for 4 projects that no longer exist: 11 GB. Xcode rebuilds it on the next build.
> - 9 Docker images that no container uses: 14 GB. Re-downloaded on the next `docker pull`.
> - 2 Hugging Face models you haven't loaded since March: 6.7 GB. About 10 minutes each to re-download.
>
> I left `hoohacks-2026` alone because it has changes you never committed.

What stays and what changes:

	MVP	**After the pivot**
Customer	Students and family with messy laptops	CS students first, then working developers
Problem	Messy Downloads and Desktop	A laptop full of rebuildable dev clutter
What it can remove	Only exact duplicates (SHA-256 proof)	Exact duplicates **and** rebuildable artifacts (rebuild proof)
Where it looks	One folder; skips repos, hidden folders, and `~/Library`	Projects, their ignored build folders, and known tool caches, but never source code
Space freed	Only reported; files sit in the Dino Bin	Real: rebuildables are deleted, and the app shows the actual change on disk
Undo	Undoes the whole last run	Per item; for a rebuildable, one click runs the recorded rebuild command
Why come back	No reason once Downloads is clean	An alert at 85% full, an end-of-semester sweep, and a cleanup after each hackathon
Unchanged	Chat → plan → validator → self-correction → ask when stuck → remember → approve	Same

The rebuild proof (new validator check). An item can be deleted only if all five hold:

1. **Known artifact type:** for example `node_modules`, a `.venv` or conda environment, Xcode DerivedData, package-manager caches, the Hugging Face cache, or Docker images no container uses.
2. **Not source code:** git tracks nothing inside it, and it's in `.gitignore` or a tool's own cache folder.
3. **A rebuild recipe exists:** the lockfile or manifest is present (`package-lock.json`, `requirements.txt`, `environment.yml`, or a model ID), and we save its hash and the exact rebuild command.
4. **Nothing irreplaceable inside:** no `.env` files, keys, datasets the user made, or trained checkpoints.

5. **Inactive:** the project hasn't changed in 30 days, or the user named it.

Source code, uncommitted work, Xcode Archives, Docker volumes, and VM disks stay NEVER no matter what.

This is the MVP's duplicate proof made broader. Today the app may delete a file because it can prove an identical copy survives. After the pivot, it may also delete a folder because it can prove the recipe to rebuild it survives.

Why the LLM is still core. Without it, this is just npkill. Deleting `node_modules` is the easy part. The hard part is knowing that "I'm done with CS 3140 but keep anything on my resume" means these six projects and not those three, then explaining each item, asking when a project looks half-finished, and remembering what you always keep.

3.3 Why this is sensible

1. **It's where the space is.** CS testers had a median of 38 GB of rebuildable clutter against 1.4 GB of duplicates, and 4 of 5 had over 10 GB. Public reports match: one npkill run found 58.68 GB across 128 `node_modules` folders, and Xcode cleanup guides routinely promise 30–50 GB back.
2. **It's where testers pointed us, unprompted.** All 5 CS testers named a place the app won't go as their real problem. C1 put it best: the app protects the one folder they most want cleaned.
3. **Our trust engine carries over.** CS testers met all three trust KPIs. C2 approved large dataset deletes because the app proved the copies identical, and the pivot relies on that same kind of proof-based approval.
4. **It fixes our retention problem by design.** Dev clutter grows back with every new project, class, and hackathon. A clean Downloads folder stays clean.
5. **It fixes the MVP's two biggest value flaws.** Rebuildables are actually deleted, so space is actually freed. And the app can finally remove things that aren't duplicates, with proof.
6. **We are the customer, and we can reach more of them.** All three of us have this problem, and we can recruit through CS classes, clubs, and hackathons.
7. **There's room to grow.** SlashData estimates 48.4 million developers worldwide as of Q3 2025. Students are the beachhead, and working developers come next.
8. **The competition proves the pain but leaves a gap.** Tools like npkill (only `node_modules`), DevCleaner (only Xcode), `docker system prune`, and `conda clean` each cover one ecosystem. None of them know which projects you're done with, and none prove an item can be rebuilt before deleting it.

Why not the other directions?

Option	Why not
Same product, better hypothesis	Both value KPIs missed, and a new hypothesis doesn't free more space
Auto-file new downloads	Below our theme bar (2/10), crowded (Sparkle, Hazel), and it frees no space
Family "computer helper" (find files, scam checks)	Only 2 of 5 non-CS testers asked for it, 3 of 5 were Low on trust, and their storage is in Apple's Photos and iCloud

Main risks:

- Developers are used to free tools, and willingness to pay is unproven.
- Every ecosystem needs its own rebuild rule. Five (JavaScript, Python, Xcode, Docker, ML caches) cover 91% of what we measured, so we'd start there.
- A wrong deletion the night before a deadline is worse than a lost PDF, so anything we can't verify as rebuildable stays NEVER.

Activity 2 · Requirement Decomposition

Source: requirements/USER_STORIES.md and GitHub issues #2–#30

Team: Nate Kim, Nate Park, Wesley Chang · **Builds on:** MVP validation and pivot decision

> These stories come from the requirements in §2: the MVP requirements we're keeping, plus what the validation sessions found. The evidence column points to the validation sessions (C1–C5, N1–N5) or to a section of the report. Every story is a GitHub issue (#2–#30), and the tests for each one are listed in tdd/README.md.

---

1. How we derived the stories

1. **List the requirements and where each came from**: our original 8 criteria, or a finding in the validation sessions (§2). Then mark the core subset: what sets us apart, what's in the MVP, and where the LLM does the work.
2. **Ask who needs each requirement and why.** The answer is a persona plus the session evidence (§3). Each pair becomes a story: *As a … I want … so that …*
3. **Split until every story is small and testable (INVEST).** For example, "the rebuild proof" became one shared rule (US-05) plus one story per ecosystem (US-06 to US-10). Each ecosystem story can ship and be tested on its own.
4. **Write acceptance criteria from what went wrong in the sessions.** For example, N3 approved without reading and lost track of an essay. That became a criterion in US-18: the summary must call out files opened in the last day.
5. **Prioritize with MoSCoW against one goal:** Alpha (Release 1) must be able to run the pivot test described in §6. Anything that test doesn't need is Should, Could, or Won't.

Sizes are story points on a 1-2-3-5-8 scale, relative to each other: 2 is about an afternoon, and 8 is most of a week for one of us. Built stories are size 0, because there's no work left. **Status** says what the MVP already has: **New** (not built), **Change** (modifies something built), or **Built** (works as-is).

---

2. Requirements

Core subset

These nine requirements set Cleanasaurus apart, cover the MVP, and hold its LLM/AI integration. The full list follows.

Requirement	Sets us apart	In the MVP	Core LLM/AI	What it does
FR1 Plain English → plan		✅	✅	Claude turns a request and a file scan into a structured plan.
FR2 Validator gate	✅	✅	✅	Plain code checks every plan Claude writes before the user sees it.
FR3 Self-correction loop	✅	✅	✅	Validator errors go back to Claude, up to 3 tries.
FR4 Ask and remember		✅	✅	Claude asks instead of guessing, and saved answers go into later plans.
FR6 Rebuild proof	✅		✅	The validator rejects any deletion Claude proposes unless the folder can be proven rebuildable.
FR7 Intent → projects	✅		✅	Claude works out which projects "I'm done with CS 3140" means.
FR11 Malware accuracy		✅		Only scanners can flag a file, so the LLM can't.
FR15 Permission zones		✅		The user decides where the app may act.
FR17 Preview and approve		✅		Nothing runs until the user approves it.

How the LLM fits in: Claude writes the plan, and plain code decides whether it can run. Claude reads the request, file names and metadata, permission zones, and saved answers, then returns a plan in a fixed JSON format. It never touches a file, flags malware, or clears a folder for deletion. The validator, the malware scanners, and (after the pivot) the rebuild proof decide those. Without an API key, an offline rule-based planner stands in so the app can still be demoed.

What sets us apart: tools like npkill, DevCleaner, `docker system prune`, and `conda clean` each cover one ecosystem. None of them know which projects you're done with, and none prove an item can be rebuilt before deleting it (validation §3.3).

Functional

ID	Requirement	Source
FR1	Turn a plain-English request into a structured plan	Criterion 5 (kept)
FR2	No plan reaches the user until the validator passes it	Criterion 6 (kept)
FR3	Send validator errors back to the planner, retry up to 3 times, then show only the safe part and ask	Criterion 7 (kept)
FR4	Ask instead of guessing when something is ambiguous, and remember the answer	Criterion 7 (kept); C5, N5
FR5	Scan projects, their ignored build folders, and known tool caches	C1, C3, C5
FR6	Delete a folder only if the validator can prove it can be rebuilt (the rebuild proof)	C1–C5; validation §2.1
FR7	Map plain-English intent ("done with CS 3140") to specific projects	C1; validation §3.2

ID	Requirement	Source
FR8	Free space for real: delete rebuildables, send personal files to the macOS Trash, and show the actual change on disk	C4; validation §2.1
FR9	Undo per action; for anything deleted, rebuild it with the recorded command	C3, N3
FR10	Watch storage and offer cleanups (85% full, end of semester)	C1; repeat-use KPI; validation §3.2
FR11	Only scanners can flag a file as malware, and an unsigned app alone is "unknown," not malware	Criterion 6 (kept); C4, N4
FR12	Plan folders of any size by working in batches of 400 files	The MVP's 400-file limit
FR13	A one-sentence, plain-word summary above Approve	N2–N4
FR14	Opt-in privacy and accuracy options: a local model, and first-page reading for sorting	C4; validation §2.2
FR15	Permission zones (FREE / ASK / NEVER) that the user can edit	Criterion 6 (kept)
FR16	Opt-in usage logs and a "Go Pro" fake door, so we can measure the pivot test without sitting with testers	Pivot test (§6)
FR17	Show every action with its reason before anything runs; the user can uncheck any action, and nothing runs until they approve	Core hypothesis; validation §2.2 (7/10)

Non-functional

ID	Requirement	Target	Source
NFR1	Safety	0 unrecoverable deletions. Source code, uncommitted work, Archives, Docker volumes, and VM disks are never deleted.	Unsafe-actions KPI; rebuild proof (validation §3.2)
NFR2	Rebuild reliability	≥95% of one-click rebuilds succeed	Pivot test KPI
NFR3	Privacy	Only file names and metadata leave the laptop, never contents, unless the user opts in	C4
NFR4	Speed	Scan of home dev folders ≤60 s; validated plan ≤30 s median (today: 34 s)	Validation §2.1 (34 s), §2.2 (too slow, 2/10)
NFR5	Honest numbers	Every "freed" number is the measured change on disk	C4

---

3. Personas

P1 · Dev student (primary). "*My laptop is 95% full and I know it's my projects. I just don't know which folders are safe to delete.*"

- CS student with a 256–512 GB MacBook.
- 10–60 GB of rebuildable clutter across 20–30 class and hackathon projects, plus Docker, Xcode, or ML caches.
- Comfortable in a terminal, but doesn't want to learn five cleanup tools.

- Afraid of breaking a project the night before a deadline.
- Drawn from C1, C2, C3, C5.

P2 · Careful engineer (secondary). "*Show me the proof, not the promise.*"

- Reads every line of a plan.
- Cares about privacy and false positives.
- Will recommend the app to others if it holds up, and will warn them off if it doesn't.
- Drawn from C4.

P3 · Non-CS student (secondary). "*Just tell me what it's going to do in one sentence.*"

- Messy Downloads folder, five versions of every essay.
- Won't read a long plan.
- Trusts the app once it remembers their answers.
- Drawn from N5 and N3, who are both students.

Not a target this release: family members who aren't students. Their needs are finding a file, checking for scams, and Photos storage. Those belong to a different product (validation §3.3), so they're in §5.

---

4. User stories

Each story lists **Priority · Size · Status · Requirements · Evidence**.

Epic A · See where my space went

US-01 · Storage breakdown before anything changes

`Must · 5 · New · FR5, NFR4, NFR5 · C1, C3; validation §2.1`

As a dev student whose laptop is almost full, **I want** to see where my space went, by type and by project, before anything changes, **so that** I know whether cleaning is worth it.

- **Given** I haven't approved anything, **when** I ask "where did my space go?", **then** I see measured totals for JavaScript packages, Python environments, Xcode build data, Docker, and ML model caches, plus my 10 largest projects, and no file is modified.
- **Given** about 50 projects under my home folder, **when** the scan runs, **then** it finishes within 60 seconds and shows progress.
- **Given** macOS blocks the app from reading a folder (Documents, Desktop), **when** the scan reaches it, **then** the app names the folder and says how to grant access, instead of silently skipping it.

US-02 · Label what's rebuildable and what isn't

`Must · 3 · New · FR5, FR6, NFR1 · Validation §3.2 (rebuild proof)`

As a dev student, **I want** every item labeled "rebuildable" (and how) or "not rebuildable" (and why), **so that** I can tell clutter apart from things I'd lose.

- **Given** a `node_modules` folder with a lockfile, **then** it's labeled "Rebuildable: `npm ci`, about 1 minute."
- **Given** Xcode Archives, a VM disk, or a Docker volume, **then** it's labeled "Not rebuildable" with a reason (for example, "crash reports need it" or "may hold a database"), and it never appears in a cleanup plan.
- **Given** an item the app doesn't recognize, **then** it's labeled "Unknown" and treated as not rebuildable.

Epic B · Clean dev clutter safely

US-03 · Clean by saying what I'm done with

`Must · 8 · New · FR1, FR4, FR7 · C1; validation §3.2 example`

As a dev student, **I want** to say which classes or projects I'm done with in my own words, **so that** I don't have to hunt down each project folder myself.

- **Given** projects named `cs3140-hw1` through `cs3140-final` and `hoohacks-2026`, **when** I say "I'm done with CS 3140 and the hackathon," **then** the plan includes artifacts from only those projects, and every action names its project and reason.
- **Given** a request that could match projects in two ways, **when** the dino can't tell which I mean, **then** it asks a question with options instead of guessing.
- **Given** I name a project that changed in the last 30 days, **then** its artifacts show up unchecked, with a note that the project is still active.

US-04 · Protect the projects I care about, once

`Must · 3 · Change (extends DINO MEMORY) · FR4, FR7 · N5; validation §3.2 example`

As a dev student, **I want** to tell Cleanasaurus once which projects to always keep, **so that** it never offers to clean them again.

- **Given** I said "keep anything on my resume" and picked `capstone-team7` and `portfolio-site`, **when** I ask for any later cleanup, **then** those projects are skipped and listed as protected.
- **Given** a saved pattern like `capstone-*`, **when** a new matching project appears, **then** it's protected too.
- **Given** I remove a protected project from DINO MEMORY, **then** it can appear in plans again.

US-05 · Delete only what can be proven rebuildable

`Must · 8 · New · FR2, FR6, NFR1 · C1, C2, C5; validation §3.2`

As a dev student, **I want** Cleanasaurus to delete a folder only when it can prove I can rebuild it, **so that** freeing space never costs me work.

- **Given** a folder that is a known artifact type, is gitignored or inside a tool cache, has its lockfile or manifest present, contains no `.env` files or user data, and belongs to a project unchanged for 30 days, **when** the validator checks a plan that deletes it, **then** the action passes, and the app saves the lockfile's hash and the exact rebuild command.
- **Given** a project with uncommitted changes, **then** nothing in it is offered for deletion, and the dino says why.
- **Given** a folder where git tracks any file, or a folder with no lockfile or manifest, **when** the planner proposes deleting it, **then** the validator rejects the action and sends the reason back to the planner.

US-06 · JavaScript packages

`Must · 5 · New · FR6, NFR5 · C1; validation §2.1 (2 of 5 had it)`

As a dev student who builds web apps, **I want** unused `node_modules` folders and package-manager caches cleaned, **so that** I get back a common kind of dev clutter.

- **Given** `node_modules` next to `package-lock.json`, `yarn.lock`, or `pnpm-lock.yaml`, **then** the recorded rebuild command is `npm ci`, `yarn install --frozen-lockfile`, or `pnpm install --frozen-lockfile`.
- **Given** a pnpm project, **then** the space shown is what deleting it actually frees, without double-counting files that are hard-linked to the shared store.
- **Given** the npm, Yarn, or pnpm cache, **then** clearing it is offered as one item with its measured size.

US-07 · Python environments

`Must · 5 · New · FR6 · Validation §2.1 (2 of 5 had it)`

As a dev student who does data science, **I want** old virtual environments and conda environments cleaned, **so that** I stop paying gigabytes for classes I've finished.

- **Given** a `.venv` whose project has `requirements.txt`, `uv.lock`, or `poetry.lock`, **then** it's offered with the matching rebuild command.
- **Given** a conda environment with no `environment.yml`, **then** before offering it, the app exports the environment into its own data folder and records that export as the rebuild recipe.
- **Given** an environment changed in the last 30 days, **then** it isn't offered unless I name it.

US-08 · Xcode build data and simulators

`Must · 5 · New · FR6, FR15 · C3`

As a dev student who builds iOS apps, **I want** build data for old projects and simulators I no longer use cleaned, **so that** I get back the tens of gigabytes Xcode hoards.

- **Given** DerivedData folders for projects that no longer exist or haven't been built in 30 days, **then** they're offered as rebuildable ("Xcode rebuilds it on the next build").

- **Given** simulator runtimes unused for 180 days, **then** they're offered with their size and a note that they re-download from Xcode Settings.
- **Given** any other folder in `~/Library`, including Xcode Archives, **then** it stays NEVER. Only these named Xcode folders are opened up.

US-09 · Docker images and build cache

`Must · 5 · New · FR6, NFR1, NFR5 · Validation §2.1 (3 of 5 had it)`

As a dev student who uses Docker, **I want** images that no container uses and old build cache removed, **so that** Docker Desktop stops eating 20 GB.

- **Given** images that no container (running or stopped) uses, **then** they're offered with their size and the note "re-downloads on the next pull."
- **Given** Docker volumes, **then** they're never offered.
- **Given** Docker isn't running, **then** the dino says so and skips Docker, instead of failing the whole plan.
- **Given** the change on disk is smaller than the space freed inside Docker, **then** both numbers are shown.

US-10 · ML model caches

`Must · 5 · New · FR6 · C2`

As a dev student who does ML, **I want** to see which downloaded models I haven't used lately and how long each would take to re-download, **so that** I can drop the big ones without worrying.

- **Given** Hugging Face or PyTorch cache entries, **then** each one shows the model name, size, last-used date, and estimated re-download time.
- **Given** a model unused for 30 days, **then** it's offered. A model used more recently is offered only if I name it.
- **Given** a model I trained or saved myself (anything outside the download cache), **then** it's never offered.

US-11 · Clear reasons when the validator says no

`Must · 3 · Change · FR2, FR3 · Validation §2.1 (validator caught 20 errors)`

As a careful engineer, **I want** to see exactly which safety check stopped an action, **so that** I can trust that the checks are real.

- **Given** the validator rejects an action, **then** the error goes back to the planner, up to 3 tries. *(Built)*
- **Given** it still fails after 3 tries, **then** only the safe part is shown and the dino asks. *(Built)*
- **Given** a rebuild-proof failure, **then** the Safety Log names which of the five conditions failed, for example "no lockfile in `hoohacks-2026`." *(New)*

Epic C · Get it back, and count it honestly

US-12 · Show the real space freed

`Must · 3 · Change · FR8, NFR5 · C4`

As a careful engineer, **I want** the "freed" number to be the real change on my disk, **so that** I can believe what the app tells me.

- **Given** a plan has run, **then** FREED shows the measured change in free disk space, not the total size of the files touched.
- **Given** personal files went to the Trash, **then** FREED notes "frees up when you empty the Trash."
- **Given** the estimate and the measured change differ by more than 10%, **then** both are shown.

US-13 · Removed personal files go to the macOS Trash

`Should · 2 · Change · FR8 · Validation §2.1 (Dino Bin frees nothing)`

As a non-CS student, **I want** removed files to go to the normal Trash, **so that** I know where to look if I want one back.

- **Given** a duplicate is removed, **then** it appears in the Finder Trash under its original name.
- **Given** I undo the action, **then** the file returns to its original folder.
- **Given** a file is quarantined for malware, **then** it still goes to the app's quarantine folder, because quarantine means isolating a file, not throwing it away.

US-14 · Undo one action

`Must · 3 · Change · FR9 · C3, N3`

As a dev student, **I want** to undo a single action instead of the whole run, **so that** I can fix one mistake without losing the rest.

- **Given** a finished run, **then** each action has its own Undo, and "Undo all" is still there.
- **Given** I undo one action, **then** every other action stays done.
- **Given** I undo a move whose original folder has since been deleted, **then** the folder is recreated.

US-15 · One-click rebuild

`Must · 5 · New · FR9, NFR2 · Pivot test KPI (rebuilds ≥95%)`

As a dev student, **I want** one click to rebuild something Cleanasaurus deleted, using the exact command it recorded, **so that** undo still works when the files are truly gone.

- **Given** a deleted `node_modules`, **when** I click Rebuild, **then** the app runs the recorded command in that project and shows success or the error.

- **Given** the lockfile changed after the deletion, **then** the app warns that the rebuild will follow the new lockfile.
- **Given** a rebuild fails, **then** the app shows the command so I can run it myself, and records the failure (we track this against the 95% target).

Epic D · Stay clean

US-16 · Storage watch

`Must · 5 · New · FR10 · C1; repeat-use KPI; validation §3.2`

As a dev student who forgets about cleanup until it's an emergency, **I want** a heads-up when my disk passes 85% full, with how much I could get back, **so that** I clean up before "Storage Almost Full."

- **Given** my disk passes 85% full, **then** I get one notification, such as "You're 87% full. I can get you about 18 GB back." It opens the plan preview and never runs anything on its own.
- **Given** I dismiss it, **then** it stays quiet until the disk fills another 5% or 7 days pass.
- **Given** I turn the watch off, **then** I get no notifications.

US-17 · End-of-semester sweep

`Could · 3 · New · FR7, FR10 · Validation §3.2 (end-of-semester sweep)`

As a dev student finishing a semester, **I want** the dino to offer to clean up that semester's class projects, **so that** old classes don't pile up year after year.

- **Given** the semester-end date I set has passed, **then** the dino offers a sweep of class projects unchanged for 30 days.
- **Given** I accept, **then** the sweep is an ordinary plan: nothing runs until I approve it.

Epic E · Trust and clarity

US-18 · One-sentence summary above Approve

`Must · 2 · Change · FR13 · N2–N4`

As a non-CS student who won't read 40 lines, **I want** a one-sentence summary of what will change, right above Approve, **so that** I know what I'm agreeing to.

- **Given** any plan, **then** one sentence above Approve says what changes, such as "Delete 6 rebuildable folders (12 GB) and move 30 files into 4 folders. Nothing outside Downloads changes."
- **Given** the plan moves a file I opened in the last 24 hours, **then** the summary names it ("includes `final_essay.docx`, opened today").
- **Given** any plan text, **then** it uses plain words: "Trash," not "Dino Bin."

US-19 · Unsigned isn't malware

`Must · 2 · Change · FR11 · C4, N4`

As a careful engineer who builds my own apps, **I want** unsigned apps labeled "unknown" instead of quarantined, **so that** my own projects aren't treated like malware.

- **Given** an unsigned app with no other evidence, **then** it's shown as "Unknown: not signed by a verified developer," with no quarantine action.
- **Given** an unsigned app that also matches a malware signature or a pirated-software name, **then** quarantine is still proposed.
- **Given** a quarantine is proposed, **then** the reason is in plain words with a clear recommendation, for example "Likely unwanted software. Quarantine it."

US-20 · Keep even file names on my laptop

`Could · 8 · New · FR14, NFR3 · C4`

As a careful engineer, **I want** to run the planner on a local model, **so that** not even my file names leave my laptop.

- **Given** local mode is on, **then** the app makes no network request during scan, plan, or validation (checked by a test).
- **Given** local mode is on, **then** the validator and the rebuild proof work exactly the same.
- **Given** I turn local mode on, **then** the app says once that plans may be slower or less accurate.

US-21 · Opt-in usage logs and a "Go Pro" fake door

`Must · 3 · Change · FR16, NFR3 · Pivot test (§6)`

As a Cleanasaurus team member, **I want** opt-in logs of plans, approvals, undos, rebuilds, and "Go Pro" clicks, **so that** I can measure the pivot test without sitting next to testers.

Status: the opt-in log, its first-launch prompt, the USAGE LOG panel, and EXPORT are already built for the MVP validation. Still to do: log rebuilds, and add the "Go Pro" fake door.

- **Given** a tester opts in, **then** we log counts, sizes, categories, and timings, but never file names or paths.
- **Given** a tester clicks "Go Pro," **then** they see "Pro isn't available yet. Want us to email you when it is?" and the click is logged.
- **Given** a tester opts out at any time, **then** logging stops and their log file is deleted.

Epic F · Personal files (secondary)

US-22 · One request for a huge folder

`Should · 3 · Change · FR12, NFR4 · The MVP's 400-file limit`

As a student with thousands of files in Downloads, **I want** one request to cover the whole folder, **so that** I don't have to ask seven times.

- **Given** 2,800 files, **when** I ask once, **then** the app plans in batches of 400 or fewer, validates each batch, and shows one combined plan.
- **Given** one batch still fails after 3 tries, **then** the other batches still show, and the dino asks about the one that failed.

US-23 · Sort by the first page (opt-in)

`Could · 5 · New · FR14, NFR3 · Validation §2.2 (sort by more than file names, 1/10)`

As a student whose lecture files have useless names, **I want** to let the app read the first page of a PDF to find its course code, **so that** "Lecture 7.pdf" lands in the right class.

- **Given** I haven't opted in, **then** only file names are used, as they are today.
- **Given** I opt in for PDFs in Downloads, **then** only the first page's text is read, and the reason shows what was found ("course code on page 1: CS 2150").
- **Given** no course code is found, **then** the file goes to Unsorted, or the dino asks.

Epic G · Already built in the MVP

These work today, so they're size 0. They're part of Alpha, and any change we make has to keep them working.

US-24 · Ask in plain English and get a plan

`Must · 0 · Built · FR1 · 7/10 ran a plan`

As a non-CS student, **I want** to ask in plain English and get a concrete plan, **so that** I don't have to sort files myself.

- **Given** an API key, **when** I send a request, **then** Claude returns a plan in a fixed JSON format, and only file names and metadata leave my laptop.
- **Given** no API key, **then** the app shows DEMO, and an offline planner makes the plan.
- **Given** I ask "Free up space," **then** the plan removes only exact duplicates and keeps one copy of each.
- **Given** I ask "Sort my school files by class," **then** the plan moves lecture PDFs into a folder for each class.

US-25 · See every action before anything runs

`Must · 0 · Built · FR2, FR17 · 7 of 10 testers`

As a careful engineer, **I want** to see every action with its reason and uncheck any of them, **so that** I stay in control.

- **Given** a plan passes the validator, **then** every action is listed with its reason and a checkbox.
- **Given** I uncheck actions and press Approve, **then** the app re-checks the actions I kept and runs only those.
- **Given** I haven't pressed Approve, **then** no file changes.
- **Given** a newer plan exists, **then** the older plan can't be approved.

US-26 · Set folders to FREE, ASK, or NEVER

`Must · 0 · Built · FR15 · No tester was shown an action touching a NEVER zone (validation §2.1)`

As a dev student, **I want** to set folders to FREE, ASK, or NEVER, **so that** some places are always off-limits.

- **Given** a folder in a NEVER zone, **when** I ask for a plan, **then** the dino says why it won't touch the folder, shows no plan, and no file changes.
- **Given** a move, rename, or duplicate removal in an ASK zone, **then** it's marked ASK FIRST and starts unchecked.
- **Given** I press a zone button for the current folder, **then** the zone is saved and applies to the next plan.
- **Given** a folder inside a code project (git repo), **then** it's NEVER.

US-27 · Get asked instead of guessed at

`Must · 0 · Built · FR4 · C5, N5`

As a non-CS student, **I want** to be asked about anything unclear, and never asked the same thing twice, **so that** the app doesn't guess with my files or make me repeat myself.

- **Given** files that look like versions of each other, **then** the dino asks with options and leaves those files out of the plan.
- **Given** a plan includes a file over 1 GB, **then** the dino asks if I'm sure.
- **Given** I answer a question, **then** the answer is saved to DINO MEMORY, a new plan follows, and the same question isn't asked again.
- **Given** I remove an answer from DINO MEMORY, **then** the app no longer uses it.

US-28 · See the safety checks at work

`Must · 0 · Built · FR2, FR3 · The validator caught 20 errors, and no unsafe action reached a tester (validation §2.1)`

As a careful engineer, **I want** a Safety Log of every check, **so that** I can see the validator working.

- **Given** a try passes, **then** the log shows its number and PASS.
- **Given** a try fails, **then** the log shows how many errors it had and the first one, and the errors go back to the planner.
- **Given** a plan runs or is undone, **then** the log shows how many actions ran or steps were undone.
- **Given** a scan finishes, **then** the log names the malware engine it used.

US-29 · Malware checks come from real scanners

`Must · 0 · Built · FR11 · No tester was shown an unflagged file for quarantine (validation §2.1); C4's false flag led to US-19`

As a careful engineer, **I want** downloads checked for malware by real scanners, never by the AI's opinion, **so that** the LLM's guess can't quarantine a file or clear one.

- **Given** a scan, **then** only ClamAV (if installed) and the built-in checks can flag a file.
- **Given** a plan quarantines a file no scanner flagged, **then** the validator rejects that action.
- **Given** a file is flagged, **then** the dino names it after the scan, and a plan that leaves it in place adds a warning to the Safety Log.
- **Given** a file is quarantined, **then** Undo moves it back to its original folder.

---

5. Won't do this release

These came up in the sessions. We're saying no for now on purpose.

Request	Who asked	Why not now
Find a downloaded form by asking	N1	Family segment is deprioritized; this is a search product
Scam checks for email attachments	N4	A different product; needs email access
Move old semesters to an external drive or cloud	N5	Useful, but the pivot test doesn't need it. First in line for Release 2.
Delete games and apps I don't use	N3	Uninstalling apps is a different safety problem
Auto-file new downloads	Two testers (validation §2.2)	Below our theme bar (2/10); revisit after the storage watch ships
Unity `Library/` folders	C5	The smallest category we measured (18 GB); next after the five main ecosystems
Windows	No tester asked	macOS first, as planned

---

6. Release plan

Alpha (Release 1) goal: run the pivot test: 20 CS students from outside our friend group, using the app unsupervised, measured against the KPIs in §7.

Priority	Stories	Points
Already built (Alpha)	US-24–US-29	0
Must (Alpha)	US-01–12, US-14–16, US-18, US-19, US-21	78
Should	US-13, US-22	5

Priority	Stories	Points
Could	US-17, US-20, US-23	16
Total	**29 stories**	**99**

Sprint plan. This assumes the three of us finish about 20 points a week. That's a guess; after Sprint 1, we'll replace it with our real pace.

Sprint	Stories	Points	Done means
1 · Foundation	US-01 breakdown, US-02 labels, US-05 rebuild proof, US-12 real freed	19	We can show a tester their clutter and prove what's safe
2 · First cleanups	US-06 JavaScript, US-07 Python, US-03 clean by intent, US-11 validator reasons	21	A tester can say "I'm done with CS 3140" and get real space back
3 · Get it back	US-15 rebuild, US-14 per-action undo, US-04 protect projects, US-18 summary, US-19 unsigned	15	Every deletion can be rebuilt, and every plan is readable in one sentence
4 · Coverage and test kit	US-08 Xcode, US-09 Docker, US-10 ML caches, US-16 storage watch, US-21 logs and fake door	23	All five ecosystems are covered, and the pivot test can run unsupervised

Dependencies: US-05 comes before US-06 to US-10 and US-15. US-01 comes before US-03. US-21 must be done before the test starts.

Definition of done (every story):

- Every acceptance criterion passes, as an automated test wherever possible. Every NEVER case in the rebuild proof has a test that proves it gets rejected.
- `npm test` and `npm run test:ui` pass.
- Demo mode still works offline.
- Any user-facing text uses plain words (US-18).

---

7. Traceability

Requirement → stories

Requirement	Stories
FR1 Plain English → plan	US-03, US-24
FR2 Validator gate	US-05, US-11, US-25, US-28
FR3 Self-correction loop	US-11, US-28
FR4 Ask and remember	US-03, US-04, US-27
FR5 Project-aware scan	US-01, US-02
FR6 Rebuild proof	US-02, US-05, US-06, US-07, US-08, US-09, US-10
FR7 Intent → projects	US-03, US-04, US-17

Requirement	Stories
FR8 Real space freed	US-12, US-13
FR9 Per-action undo and rebuild	US-14, US-15
FR10 Storage watch and sweeps	US-16, US-17
FR11 Malware accuracy	US-19, US-29
FR12 Large folders in batches	US-22
FR13 One-sentence summary	US-18
FR14 Privacy and accuracy options	US-20, US-23
FR15 Permission zones	US-08, US-26
FR16 Usage logs and fake door	US-21
FR17 Preview and approve	US-25
NFR1 Safety	US-02, US-05, US-09
NFR2 Rebuild reliability	US-15
NFR3 Privacy	US-20, US-21, US-23
NFR4 Speed	US-01, US-22
NFR5 Honest numbers	US-01, US-06, US-09, US-12

Pivot test KPI → stories it depends on

KPI	Target	Stories
Testers with ≥10 GB of rebuildable clutter	≥70%	US-01, US-06–US-10
Unsupervised first-session approval of a plan freeing ≥10 GB	≥60%	US-02, US-03, US-05, US-18
Real space freed (median)	≥10 GB	US-06–US-10, US-12
One-click rebuilds that work	≥95%	US-15
Repeat use within 30 days	≥40%	US-04, US-16
Unrecoverable deletions	0	US-02, US-05
"Go Pro" fake-door clicks / pre-orders	≥15% / ≥5%	US-21

GitHub issues

Every user story above is a GitHub issue, labeled by priority (MoSCoW), Alpha scope, and status. The six stories the MVP already delivers are closed as built.

Issue	Story	Labels	State
#2	US-01 · Storage breakdown before anything changes	alpha, priority: must, status: new	Open
#3	US-02 · Label what's rebuildable and what isn't	alpha, priority: must, status: built	Closed
#4	US-03 · Clean by saying what I'm done with	alpha, priority: must, status: new	Open

Issue	Story	Labels	State
#5	US-04 · Protect the projects I care about, once	alpha, priority: must, status: change	Open
#6	US-05 · Delete only what can be proven rebuildable	alpha, priority: must, status: built	Closed
#7	US-06 · JavaScript packages	alpha, priority: must, status: built	Closed
#8	US-07 · Python environments	alpha, priority: must, status: built	Closed
#9	US-08 · Xcode build data and simulators	alpha, priority: must, status: new	Open
#10	US-09 · Docker images and build cache	alpha, priority: must, status: new	Open
#11	US-10 · ML model caches	alpha, priority: must, status: new	Open
#12	US-11 · Clear reasons when the validator says no	alpha, priority: must, status: built	Closed
#13	US-12 · Show the real space freed	alpha, priority: must, status: built	Closed
#14	US-13 · Removed personal files go to the macOS Trash	priority: should, status: change	Open
#15	US-14 · Undo one action	alpha, priority: must, status: change	Open
#16	US-15 · One-click rebuild	alpha, priority: must, status: new	Open
#17	US-16 · Storage watch	alpha, priority: must, status: new	Open
#18	US-17 · End-of-semester sweep	priority: could, status: new	Open
#19	US-18 · One-sentence summary above Approve	alpha, priority: must, status: change	Open
#20	US-19 · Unsigned isn't malware	alpha, priority: must, status: change	Open
#21	US-20 · Keep even file names on my laptop	priority: could, status: new	Open
#22	US-21 · Opt-in usage logs and a "Go Pro" fake door	alpha, priority: must, status: change	Open
#23	US-22 · One request for a huge folder	priority: should, status: change	Open
#24	US-23 · Sort by the first page (opt-in)	priority: could, status: new	Open
#25	US-24 · Ask in plain English and get a plan	alpha, priority: must, status: built	Closed
#26	US-25 · See every action before anything runs	alpha, priority: must, status: built	Closed
#27	US-26 · Set folders to FREE, ASK, or NEVER	alpha, priority: must, status: built	Closed
#28	US-27 · Get asked instead of guessed at	alpha, priority: must, status: built	Closed
#29	US-28 · See the safety checks at work	alpha, priority: must, status: built	Closed
#30	US-29 · Malware checks come from real scanners	alpha, priority: must, status: built	Closed

Activity 3 · AI-Assisted Test-Driven Development

Source: tdd/README.md, tdd/01-red-run.txt, tdd/02-green-run.txt

Team: Nate Kim, Nate Park, Wesley Chang · **Branch:** `alpha-dev` · **Stories and issues:** requirements/USER_STORIES.md, GitHub issues #2–#30

1. How we ran it

1. **Tests first.** For every open GitHub issue we wrote tests from the story's acceptance criteria before any production code. Claude generated the first drafts, and we inspected and refined them (see the review log). All story tests live in `app/test/stories/`, one file per issue, named `usNN-*.test.js`.
2. **Confirm they fail.** We committed the tests on their own and ran them. The run is saved in `01-red-run.txt`.
3. **Build the Alpha core until green.** We built the stories the pivot depends on (rebuild proof, labels, JavaScript and Python cleanup, honest space freed, validator reasons) one at a time, rerunning the suite after each. The final run is saved in `02-green-run.txt`.
4. **Everything else stays red, on purpose.** Tests for stories we haven't built yet run on every `npm test` and are marked `todo` with their issue number, so they're visible but don't block the build. When someone starts one of those stories, they remove `todo`, watch it fail, then build it.
5. **CI.** `.github/workflows/test.yml` runs `npm test` on every push, so each red and green step is recorded on GitHub.

The commits, in order

Commit	What it shows	Result
`c1c337d`	Tests for every open issue, **no code**	🔴 11 failing (core stories), 46 todo, 25 existing pass
`d2f885d`	Rebuild proof, labels, JS and Python detection, plus one edge-case test added during review	🟢 US-02, US-06, US-07 pass
`454d384`	Validator, executor, engine, and planner use the proof; FREED is measured on disk	🟢 US-05, US-11, US-12 pass; tests caught bug REG-1
`86c2846`	CI workflow and a dev-clutter demo folder (`npm run dev-demo`)	🟢
`14138e7`	Regression REG-2 (found in a screenshot), test written first	🔴 → 🟢
`607919a`	Wording: build folders are rebuilt, not undone	🟢 UI test 14/14

Totals

	Before (`c1c337d`)	**After (`607919a`)**
Backend tests passing	25	**62**
Failing	11	**0**

	Before (`c1c337d`)	After (`607919a`)
Todo (written, story not built yet)	46	46
UI end-to-end test (clicks the real app)	14 steps	**14/14 pass**

Run them: `cd app && npm test` and `npm run test:ui`.

2. Every GitHub issue and its tests

Issue	Story	Priority	Tests	Status
#2	US-01 Storage breakdown	Must	`us01-storage-breakdown.test.js` (3)	🔴 todo
#3	US-02 Label rebuildable vs. not	Must	`us02-labels.test.js` (5)	🟢 pass
#4	US-03 Clean by intent	Must	`us03-clean-by-intent.test.js` (3)	🔴 todo
#5	US-04 Protect projects	Must	`us04-protect-projects.test.js` (3)	🔴 todo
#6	US-05 Rebuild proof	Must	`us05-rebuild-proof.test.js` (12)	🟢 pass
#7	US-06 JavaScript packages	Must	`us06-javascript.test.js` (6)	🟢 pass*
#8	US-07 Python environments	Must	`us07-python.test.js` (5)	🟢 pass*
#9	US-08 Xcode	Must	`us08-xcode.test.js` (3)	🔴 todo
#10	US-09 Docker	Must	`us09-docker.test.js` (4)	🔴 todo
#11	US-10 ML caches	Must	`us10-ml-caches.test.js` (3)	🔴 todo
#12	US-11 Validator reasons	Must	`us11-validator-reasons.test.js` (3)	🟢 pass
#13	US-12 Real space freed	Must	`us12-real-freed.test.js` (4), UI step 5	🟢 pass
#14	US-13 macOS Trash	Should	`us13-macos-trash.test.js` (2)	🔴 todo
#15	US-14 Undo one action	Must	`us14-undo-one.test.js` (3)	🔴 todo
#16	US-15 One-click rebuild	Must	`us15-rebuild.test.js` (3)	🔴 todo
#17	US-16 Storage watch	Must	`us16-storage-watch.test.js` (3)	🔴 todo
#18	US-17 Semester sweep	Could	`us17-semester-sweep.test.js` (2)	🔴 todo
#19	US-18 One-sentence summary	Must	`us18-summary.test.js` (3)	🔴 todo
#20	US-19 Unsigned isn't malware	Must	`us19-unsigned.test.js` (2)	🔴 todo
#21	US-20 Local model	Could	`us20-local-model.test.js` (2)	🔴 todo
#22	US-21 Logs and Go Pro	Must	`usage-log.test.js` (8, built part), `us21-logs-go-pro.test.js` (2)	🟡 log built, rest todo
#23	US-22 Huge folders	Should	`us22-huge-folder.test.js` (2)	🔴 todo
#24	US-23 First-page sorting	Could	`us23-first-page.test.js` (3)	🔴 todo

Issue	Story	Priority	Tests	Status
#25	US-24 Plain English → plan	Built	`engine.test.js` (demo plan), `minimal.test.js` MIN-1, MIN-2	🟢 pass
#26	US-25 Preview and approve	Built	`engine.test.js` (newest plan only), MIN-6, UI steps 4–8	🟢 pass
#27	US-26 Permission zones	Built	MIN-7, `engine.test.js` (NEVER refusal, home git repo), UI step 7	🟢 pass
#28	US-27 Ask and remember	Built	MIN-8, `engine.test.js` (answers remembered), UI steps 8–9	🟢 pass
#29	US-28 Safety checks visible	Built	`engine.test.js` (rejects unsafe plans, retries)	🟢 pass
#30	US-29 Malware from real scanners	Built	MIN-3, `engine.test.js` (flags test malware)	🟢 pass
—	Regressions	—	`regressions.test.js` REG-1, REG-2	🟢 pass

* The acceptance criteria pass at the unit level. Two pieces are built but not yet wired into plans: package-manager caches (US-06) and the conda export (US-07). They're listed as the next step in §5.

3. What the core does now (the green part)

- **Rebuild proof** (`backend/devclutter.js`): a folder can be deleted only if every check passes:
  - Known type (`node_modules`, a Python venv, a conda env).
  - Not source: git tracks nothing inside it, and it's gitignored.
  - A lockfile or requirements file exists. Its SHA-256 hash and the exact rebuild command are saved.
  - No `.env`, key, or certificate files inside.
  - The project is unchanged for 30 days, or the user named it.
  - No uncommitted work.
- **Every failure names its condition**, for example "no lockfile (package-lock.json, yarn.lock, or pnpm-lock.yaml) in hoohacks-2026". The reason goes back to the planner and shows in the Safety Log.
- **Real space freed**: proven folders are really deleted, and FREED is the measured change in free disk space. Duplicates in the Dino Bin are reported as "frees up when you empty it", which fixes the "12.4 GB reported, 0 bytes freed" finding from validation.
- **Honest sizes**: files hard-linked to pnpm's shared store aren't counted as freed.
- **Demo:** `npm run dev-demo` builds old projects plus every case the proof must refuse. On it, the planner proposes 6 folders, the validator rejects 3 (uncommitted work, a project touched 4 days ago, a `.env` secret), and the retry passes with 3 deletions that free about 150 MB measured on disk.

4. Review log: what we changed in the AI-generated tests

Claude drafted the tests from each story's acceptance criteria. Before writing code, we checked each file against three questions: does it test the criterion and not the implementation, does it cover the NEVER cases, and does it miss an edge case?

#	Finding	Type	What we did
1	One of us (Wesley) has a git repo at the home folder (dotfiles). A naive proof would call every project "not gitignored."	Missing edge case	Added a US-05 test **before** writing the proof; the proof ignores a repo at home or above, like the zones already do
2	Generated tests only had the happy path for `node_modules`	Missing edge cases	Added: no lockfile → not rebuildable; nested `node_modules` listed once; a file hard-linked twice counted once; a venv not named `.venv`; the project folder itself as a target; NEVER zone still blocks
3	US-11 test expected exactly one failure, but deleting `package.json` in a committed repo also leaves uncommitted changes	Wrong test assumption	Test now looks for the `recipe` failure among all failures and also expects `clean_worktree`
4	MIN-1 and UI step 5 asserted that FREED grows after removing duplicates	Requirement changed	US-12 makes FREED the measured disk change, and the Dino Bin is on the same disk. Updated both to check the estimate and the "frees up when you empty it" note
5	US-11's engine test kept failing all 3 tries	**Real bug the tests found** (REG-1)	Identical source files in two code projects were offered as duplicates, rejected, and re-offered. The planner now skips NEVER zones and drops anything the validator rejected
6	A screenshot showed "6 different versions of index.js"	**Real bug** (REG-2)	Wrote REG-2 first (failed), then fixed it: source files in code projects never trigger a "versions" question
7	The todo test "US-03 ambiguous request asks" already passes by accident (the demo planner proposes nothing)	Weak test	Left as todo; it doesn't count as done until US-03 is built. Whoever builds US-03 should tighten it

Known limits of the current tests:

- The secret check is cautious: a `node_modules` that ships test certificates (`.pem`) will be refused. That's the safe direction, but it may hide real space.
- The measured FREED can be off if another app writes to the disk during a run. Tests use a fake disk to stay deterministic.
- The tests need `git`, and CI runs on macOS because the malware checks call `codesign` and `xattr`.
- The LLM path is covered by the same validator, but these tests run the offline planner, not Claude.

5. Next stories, in sprint order

1. Wire package caches (US-06) and the conda export (US-07) into plans.
2. US-01 storage breakdown, then US-03 clean by intent (needs US-01).
3. US-15 one-click rebuild and US-14 per-action undo.

4. US-18 summary and US-19 unsigned-is-unknown (small).
5. US-08 Xcode, US-09 Docker, US-10 ML caches, US-16 storage watch, US-21 Go Pro.

Appendix · The Alpha running

npm run dev-demo, offline planner



The planner proposed six build folders. The validator rejected three (uncommitted work in hoohacks-2026, a project changed 4 days ago, a .env file inside node_modules) and sent the reasons back; the second try passed with three folders, each showing its saved rebuild command.